p8106_hw3

Charlotte Zhang

2026-03-30

```
# Import data
suppressPackageStartupMessages({
  library(tidyverse)
  library(tidymodels)
  library(caret)
  library(earth)
  library(MASS)
  library(pROC)
  library(vip)
  library(pdp)
  library(car)
})
```

```
## Warning: package 'parsnip' was built under R version 4.5.2
```

```
## Warning: package 'rsample' was built under R version 4.5.2
```

```
## Warning: package 'earth' was built under R version 4.5.2
```

```
## Warning: package 'plotmo' was built under R version 4.5.2
```

```
## Warning: package 'plotrix' was built under R version 4.5.2
```

```
## Warning: package 'vip' was built under R version 4.5.2
```

```
## Warning: package 'pdp' was built under R version 4.5.2
```

```
auto <- read.csv("auto.csv")
auto <- na.omit(auto)
auto$origin <- as.factor(auto$origin)
auto$mpg_cat <- factor(auto$mpg_cat, levels = c("high", "low")) # low is my positive class
```

```
# train test split
set.seed(123)
data_split <- initial_split(auto, prop = 0.7, strata = mpg_cat)
training_data <- training(data_split)
testing_data  <- testing(data_split)
```

```r
set.seed(123)
# cross validation
ctrl <- trainControl(
  method = "cv",
  number = 10,
  summaryFunction = twoClassSummary,
  classProbs = TRUE
)
```

Problem a

```r
set.seed(123)
glm.fit <- glm(mpg_cat ~ .,
               data = training_data,
               family = binomial(link = "logit"))
summary(glm.fit)
```

```
##
## Call:
## glm(formula = mpg_cat ~ ., family = binomial(link = "logit"),
##     data = training_data)
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)  12.352520   6.399625   1.930  0.05358 .
## cylinders     0.746869   0.512396   1.458  0.14495
## displacement -0.021101   0.015252  -1.383  0.16654
## horsepower    0.054933   0.029055   1.891  0.05867 .
## weight        0.004811   0.001497   3.213  0.00131 **
## acceleration  0.108188   0.170898   0.633  0.52670
## year         -0.426376   0.090919  -4.690 2.74e-06 ***
## origin2      -1.918865   0.884138  -2.170  0.02998 *
## origin3      -1.201633   0.830839  -1.446  0.14810
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 379.84  on 273  degrees of freedom
## Residual deviance: 116.69  on 265  degrees of freedom
## AIC: 134.69
##
## Number of Fisher Scoring iterations: 8
```

```r
vif(glm.fit)
```

```
##                   GVIF Df GVIF^(1/(2*Df))
## cylinders     5.129669  1        2.264877
## displacement 10.708187  1        3.272337
```

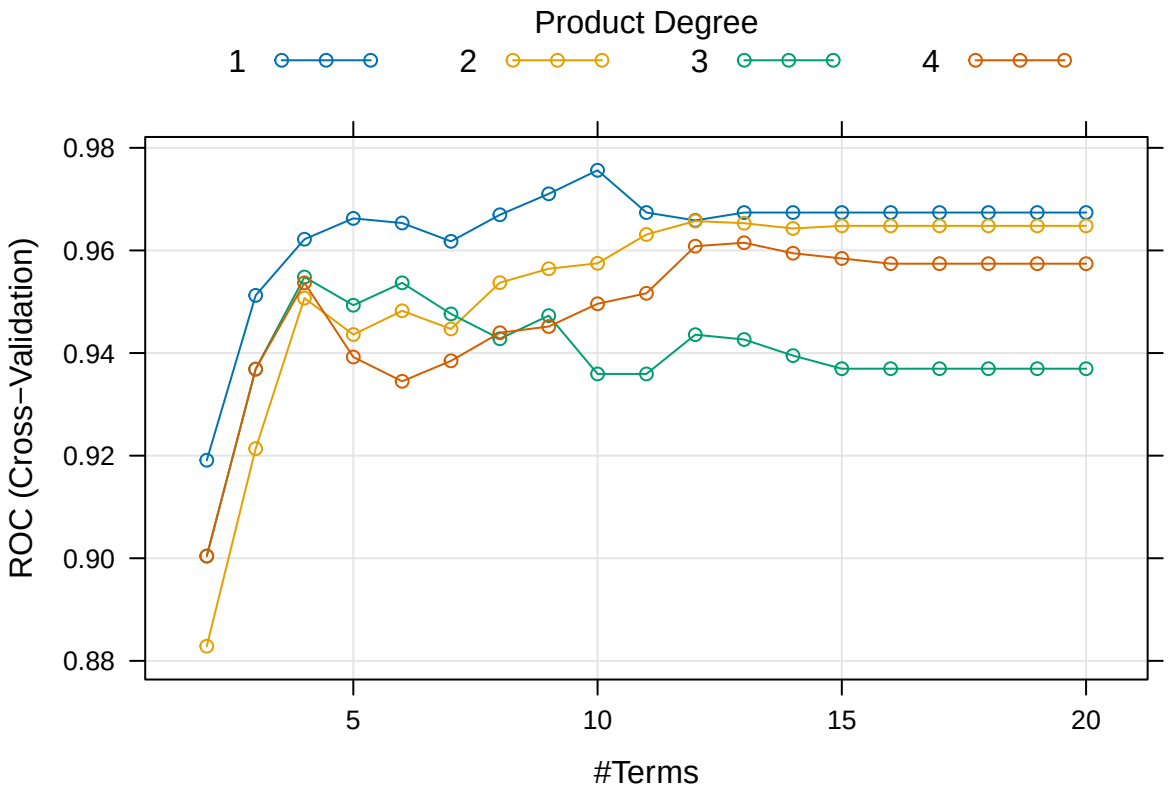
```
## horsepower    2.839744  1        1.685154
## weight        5.280190  1        2.297866
## acceleration  2.726218  1        1.651126
## year          1.883743  1        1.372495
## origin        3.091637  2        1.326011
```

Predictors with a large p-value such as cylinders, displacement, origin3, horsepower, and acceleration might be redundant. They are not statistically significant after controlling for the others, since they have a p-value greater than 0.05. This is consistent with the GVIF, where displacement (3.27), weight (2.30) and cylinders (2.26) have the largest adjusted GVIF values, possibly due to multicollinearity or overlapping information.

```
# train logistic regression model with CV
set.seed(123)
model.glm <- train(x = training_data[, names(training_data) != "mpg_cat"],
y = training_data$mpg_cat,
method = "glm",
metric = "ROC",
trControl = ctrl)
```

Problem b

```
# train MARS
set.seed(123)
model.mars <- suppressWarnings(
  train(
    x = training_data[, names(training_data) != "mpg_cat"],
    y = training_data$mpg_cat,
    method = "earth",
    tuneGrid = expand.grid(
      degree = 1:4,
      nprune = 2:20
    ),
    metric = "ROC",
    trControl = ctrl
  )
)
plot(model.mars)
```

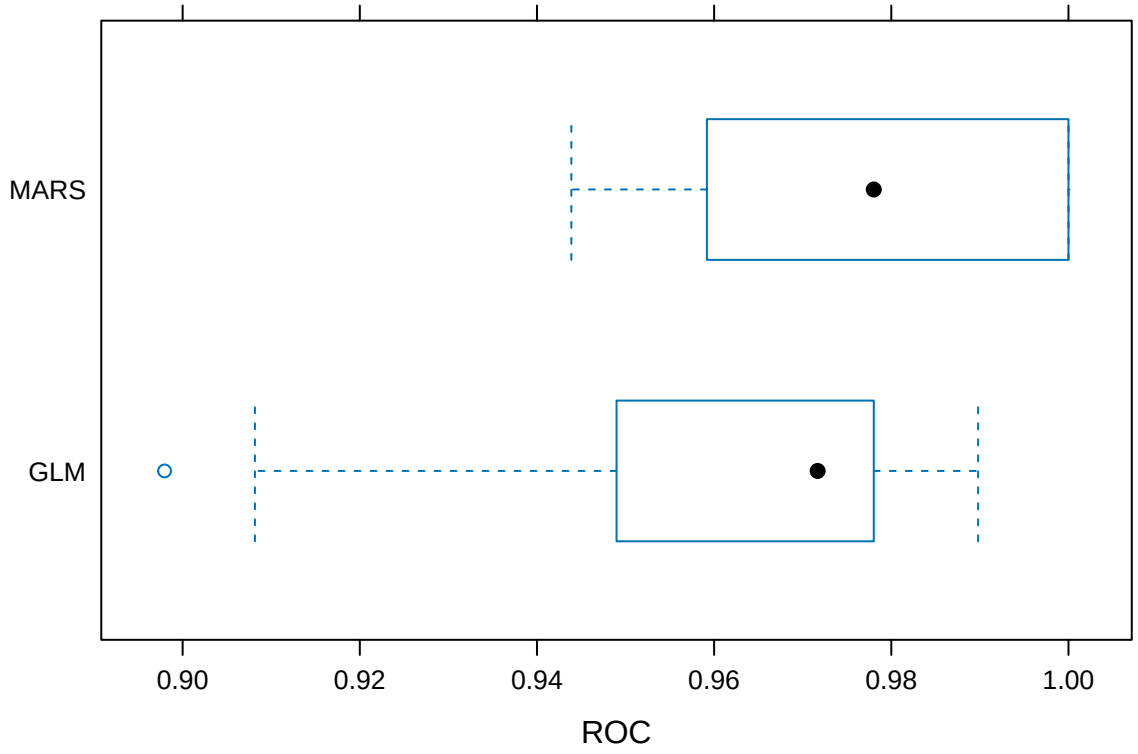



```
set.seed(123)
# compare models using resampling
res <- resamples(list(GLM = model.glm,
                      MARS = model.mars))
summary(res)
```

```
##
## Call:
## summary.resamples(object = res)
##
## Models: GLM, MARS
## Number of resamples: 10
##
## ROC
##           Min.   1st Qu.    Median      Mean   3rd Qu.      Max. NA's
## GLM  0.8979592 0.9521193 0.9716822 0.9590539 0.9780220 0.9897959    0
## MARS 0.9438776 0.9591837 0.9780220 0.9756279 0.9972527 1.0000000    0
##
## Sens
##           Min.   1st Qu.    Median      Mean   3rd Qu. Max. NA's
## GLM  0.8571429 0.8736264 0.9230769 0.9126374 0.9285714    1    0
## MARS 0.7692308 0.9230769 0.9642857 0.9329670 1.0000000    1    0
##
## Spec
##           Min.   1st Qu.    Median      Mean   3rd Qu. Max. NA's
```

```
## GLM  0.7142857 0.8571429 0.8571429 0.8769231 0.9271978    1    0
## MARS 0.7142857 0.8571429 0.8901099 0.8846154 0.9271978    1    0
```

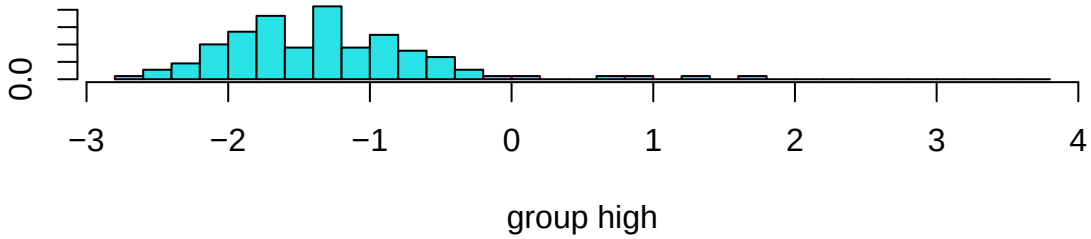
```
# boxplot of ROC performance for the 2 models
bwplot(res, metric = "ROC")
```

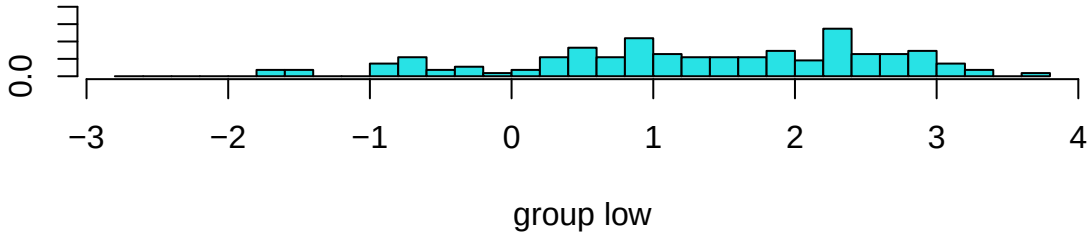


Based on the boxplots and the resample summary, MARS model (0.98 mean ROC) performs better than logistic regression (0.96 mean ROC) and it appears to have more stable performance across resamples. This comparison is made on cross-validated ROC values from the training data, and the test set ROC and AUC will be computed and plotted in part (d).

Problem c

```
set.seed(123)
lda.fit <- lda(mpg_cat ~ ., data = training_data)
# plot discriminant
plot(lda.fit)
```






```
# show coefficients
lda.fit$scaling
```

```
##                       LD1
## cylinders     0.620150994
## displacement -0.004471024
## horsepower   -0.006259062
## weight        0.001225622
## acceleration  0.010490310
## year         -0.129127247
## origin2      -0.532935359
## origin3      -0.409467160
```

The two classes are well separated along LD1. The high mpg group is mostly around at lower LD1 values, while the low mpg group is concentrated at higher LD1 values, despite some overlaps.

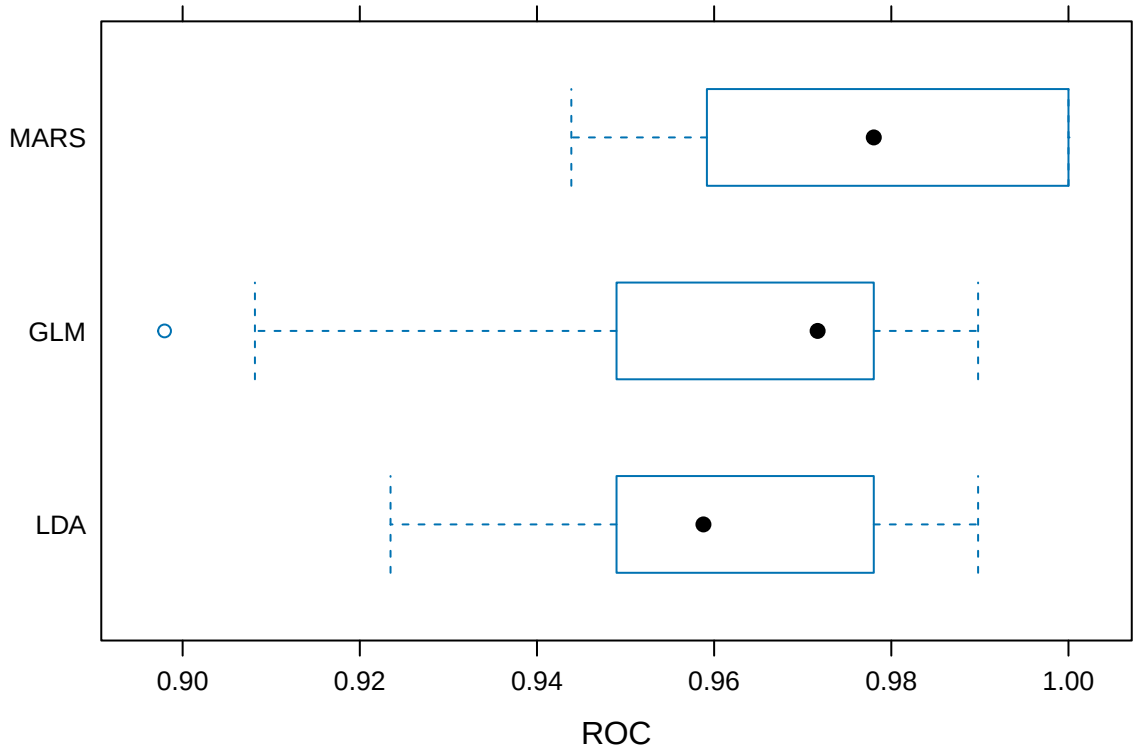
```
# Cross validation LDA
set.seed(123)
model.lda <- train(mpg_cat ~ .,
                   data = training_data,
                   method = "lda",
                   metric = "ROC",
                   trControl = ctrl)
```

Problem d

```
set.seed(123)
# compare models using resampling
res <- resamples(list(GLM = model.glm,
                      LDA = model.lda,
                      MARS = model.mars))
summary(res)
```

```
##
## Call:
## summary.resamples(object = res)
##
## Models: GLM, LDA, MARS
## Number of resamples: 10
##
## ROC
##           Min.   1st Qu.    Median      Mean   3rd Qu.      Max. NA's
## GLM  0.8979592 0.9521193 0.9716822 0.9590539 0.9780220 0.9897959    0
## LDA  0.9234694 0.9502551 0.9587912 0.9610162 0.9775993 0.9897959    0
## MARS 0.9438776 0.9591837 0.9780220 0.9756279 0.9972527 1.0000000    0
##
## Sens
##           Min.   1st Qu.    Median      Mean   3rd Qu. Max. NA's
## GLM  0.8571429 0.8736264 0.9230769 0.9126374 0.9285714    1    0
## LDA  0.8461538 0.9285714 1.0000000 0.9560440 1.0000000    1    0
## MARS 0.7692308 0.9230769 0.9642857 0.9329670 1.0000000    1    0
##
## Spec
##           Min.   1st Qu.    Median      Mean   3rd Qu. Max. NA's
## GLM  0.7142857 0.8571429 0.8571429 0.8769231 0.9271978    1    0
## LDA  0.7142857 0.8008242 0.8571429 0.8472527 0.8571429    1    0
## MARS 0.7142857 0.8571429 0.8901099 0.8846154 0.9271978    1    0
```

```
# boxplot of ROC performance for the 3 models
bwplot(res, metric = "ROC")
```



By comparing the three models using cross-validated ROC values from the training data, MARS performed the best while LDA performed the worst.

```
# Now do predictions on test data for each model
set.seed(123)
glm.pred  <- predict(model.glm,  newdata = testing_data, type = "prob")[,"low"]
mars.pred <- predict(model.mars, newdata = testing_data, type = "prob")[,"low"]
lda.pred  <- predict(model.lda,  newdata = testing_data, type = "prob")[,"low"]


roc.glm  <- roc(testing_data$mpg_cat, glm.pred)
```

```
## Setting levels: control = high, case = low
```

```
## Setting direction: controls < cases
```

```
roc.mars <- roc(testing_data$mpg_cat, mars.pred)
```

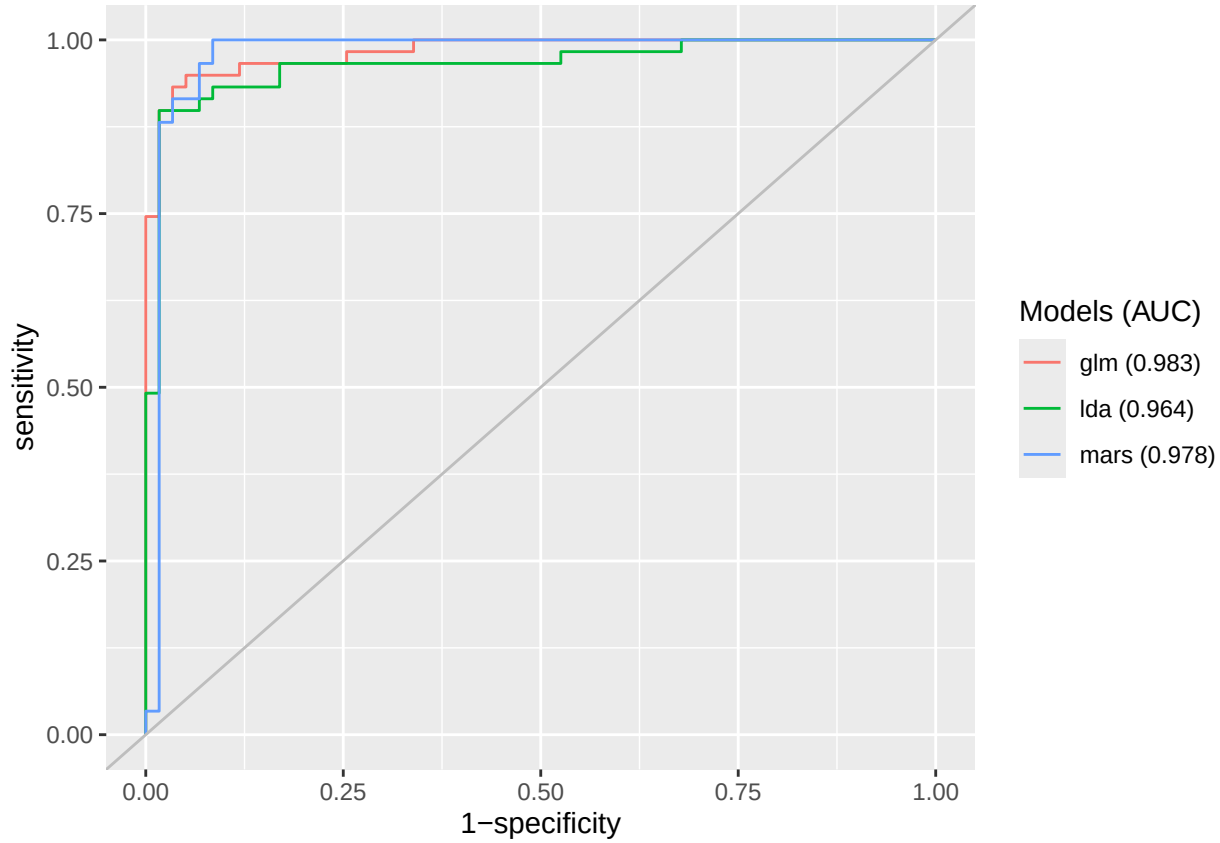
```
## Setting levels: control = high, case = low
## Setting direction: controls < cases
```

```
roc.lda  <- roc(testing_data$mpg_cat, lda.pred)
```

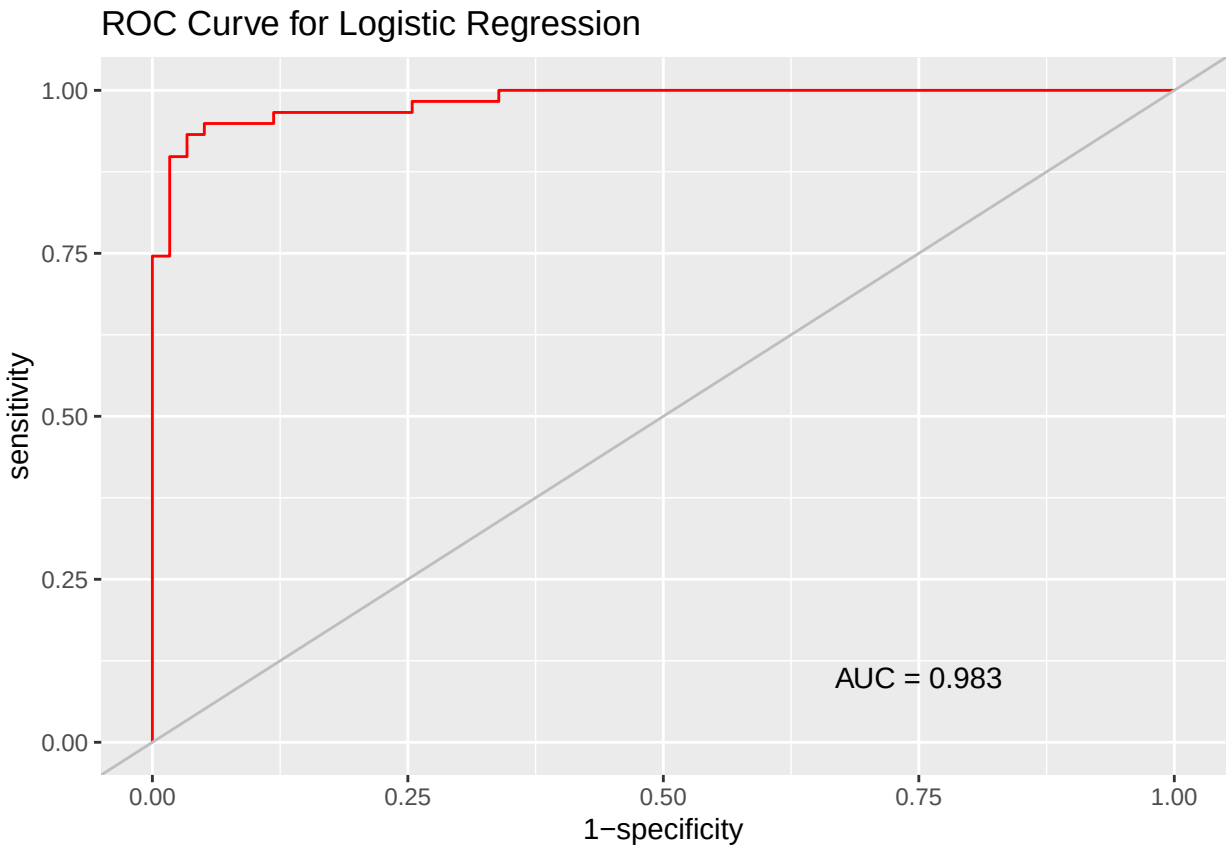
```
## Setting levels: control = high, case = low
## Setting direction: controls < cases
```

```r
auc <- c(roc.glm$auc[1], roc.lda$auc[1], roc.mars$auc[1])
```

```r
set.seed(123)
# plot ROC curves for all models
modelNames <- c("glm","lda","mars")
ggroc(list(roc.glm, roc.lda, roc.mars), legacy.axes = TRUE) +
scale_color_discrete(labels = paste0(modelNames, " (", round(auc,3),")"),
name = "Models (AUC)") +
geom_abline(intercept = 0, slope = 1, color = "grey")
```




```r
ggroc(roc.glm, legacy.axes = TRUE, color = "red") +
  geom_abline(intercept = 0, slope = 1, color = "grey") +
  labs(title = "ROC Curve for Logistic Regression") +
  annotate("text", x = 0.75, y = 0.1,
           label = paste0("AUC = ", round(roc.glm$auc, 3)))
```



While MARS performed better during cross-validation on the training data, logistic regression performed better on the test set. Since it has a test AUC of 0.983 which is higher than 0.978. Therefore, MARS did not improve prediction performance compared to logistic regression, and I would select logistic regression as the final model b/c it performs better on unseen data.

Problem e

```
final.pred <- rep("high", length(glm.pred))
final.pred[glm.pred > 0.5] <- "low"
final.pred <- factor(final.pred, levels = c("high", "low"))

confusionMatrix(data = final.pred,
                reference = testing_data$mpg_cat,
                positive = "low")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction high low
##       high   56   3
##       low     3  56
##
##                Accuracy : 0.9492
```

```
##                  95% CI : (0.8926, 0.9811)
##     No Information Rate : 0.5
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.8983
##
##  Mcnemar's Test P-Value : 1
##
##             Sensitivity : 0.9492
##             Specificity : 0.9492
##          Pos Pred Value : 0.9492
##          Neg Pred Value : 0.9492
##              Prevalence : 0.5000
##          Detection Rate : 0.4746
##    Detection Prevalence : 0.5000
##       Balanced Accuracy : 0.9492
##
##        'Positive' Class : low
##
```

I used a threshold of 0.5 which is a pretty standard cutoff for a binary classification, and it makes a good prediction because only 6 out of 118 observations were wrongly classified. Accuracy, sensitivity (TP) and specificity (TN) are all 0.9492, indicating a balanced and effective performance in identifying low and high mpg cars.